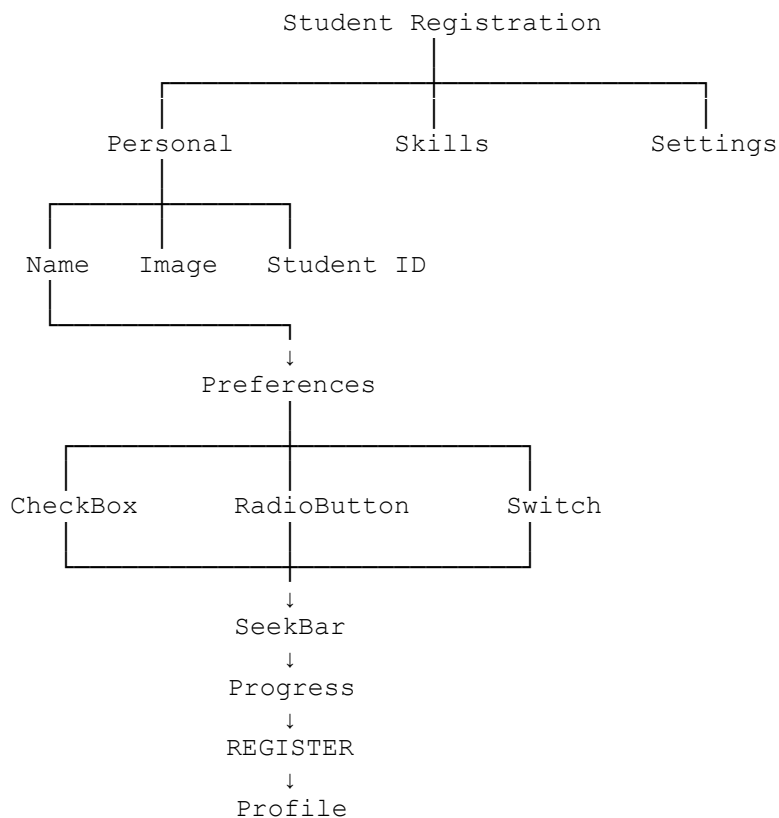


ใบงาน เรื่อง Android UI Components → Event → State → Navigation

ให้นักศึกษาศึกษาและเขียนโค้ดตามใบงานนี้เพื่อสร้างแอปพลิเคชัน **Student Registration** ซึ่งเนื้อหาในใบงานประกอบด้วย Event Handling + UI Components + Navigation โดยใบงานนี้จะนำ UI Component มารับข้อมูล, รับ Event, เปลี่ยน State, Navigate

1. เปลี่ยน Project ให้เหมาะกับ UI Components (11 ตัว)

Flow:



2. UI Components

กำหนดหน้าที่ของแต่ละตัวดังนี้

UI เดิมจากเอกสาร	Compose ที่ใช้ใน Lab	หน้าที่ใน Project
TextView	Text()	ชื่อ/คำอธิบาย
Button	Button()	Register / Save
EditText	TextField()	กรอกชื่อ
ImageView	Image()	รูปนักศึกษา
ImageButton	IconButton()	ปุ่มเปลี่ยนรูป/เมนู
CheckBox	Checkbox()	เลือกทักษะ
ToggleButton	IconToggleButton() / Toggle UI	เปิด-ปิด Favorite
RadioButton	RadioButton()	เลือกระดับการศึกษา
RadioGroup	State กลุ่มเดียวของ RadioButton	บังคับเลือกได้ 1 ตัว
Switch	Switch()	เปิด Dark Mode / Notification
ProgressBar	LinearProgressIndicator()	แสดงความสมบูรณ์ของ Profile
SeekBar	Slider()	กำหนดระดับความสนใจ

***** ชื่อ TextView, EditText, ImageView ฯลฯ เป็นศัพท์ของ Android View/XML ที่เรียนจากเอกสารเดิม แต่ใน Project นี้เราจะใช้ **Jetpack Compose equivalents** เพื่อให้เห็นว่าแนวคิด UI เดิมถูกนำมาสร้างด้วย Compose อย่างไร

3. Mockup

Mockup จะมี 4 หน้าจอ

Screen 1 — Registration

Student Register

○ PHOTO

[]

Name

Enter your name

Student ID

6612345678

Degree

○ Bachelor

○ Master

Skills

☒ Python

☒ Kotlin

☐ Java

Notifications

[ON]

Interest Level

0  100

Profile Completion: 75%



[REGISTER]

ตรงนี้นักศึกษาจะเห็น **Component** เกือบทั้งหมดในหน้าเดียว

Screen 2 — Profile

หลัง Register

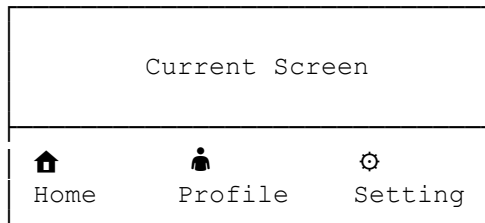
←	My Profile
PHOTO 	
Somchai Jaidee Computer Science ID: 6612345678	
Degree Bachelor	
Skills Python • Kotlin	
Interest 75%	
Notification ON	

Screen 3 — Settings

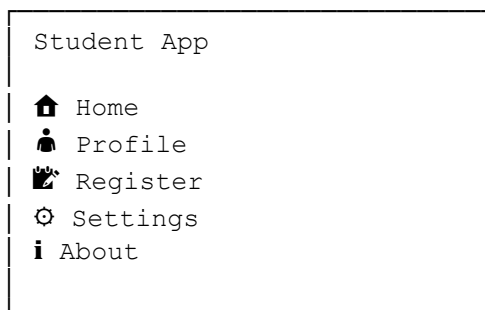
Settings	
Dark Mode	[ON]
Notifications	[ON]
Favorite	[★]
About Student Registration App	

Screen 4 — Navigation

Bottom Navigation



Navigation Drawer



4. การทำใบงาน

เพื่อให้เป็น Self-Study ตามหลัก Project-Based Learning

ดู Mockup



สร้าง UI



Run



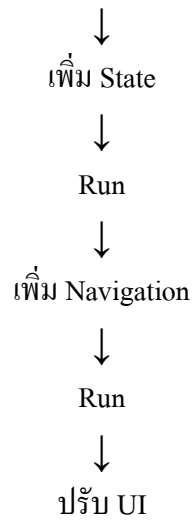
เห็นผล



เพิ่ม Event



Run



การใช้งาน UI Component (Android View แบบเดิม --> Compose)

1) TextView → Text

Step 1. สร้างชื่อหน้า

```
Text(  
    text = "Student Registration"  
)
```

ทดลองเปลี่ยน

จาก

"Student Registration"

เป็น

"My Student Profile"

2) EditText → TextField

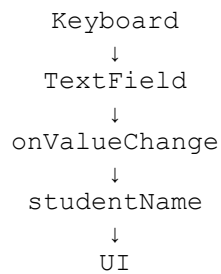
Step 2. สร้างช่องกรอกชื่อ

```
var studentName by remember {  
    mutableStateOf("")  
}
```

จากนั้น

```
TextField(  
    value = studentName,  
    onValueChange = {  
        studentName = it  
    },  
    label = {  
        Text("Enter your name")  
    }  
)
```

Flow



ให้พิมพ์

Somchai Jaidee

แล้วให้ชื่อไปแสดงด้านล่าง

```
Text(  
    text = "Hello $studentName"  
)
```

3) ImageView → Image

Step 3. แสดงรูปนักศึกษา

```
Image(  
    painter = painterResource(  
        id = R.drawable.student  
    ),  
    contentDescription = "Student Photo",  
    modifier = Modifier.size(120.dp)  
)
```

นักศึกษาต้องทำ

1. เตรียมรูป
2. ใส่ใน drawable
3. เรียกด้วย R.drawable.student
4. Run

4) ImageButton → IconButton

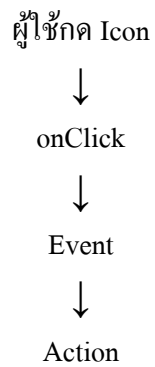
นี่จะเป็นสิ่งที่ทำให้ Lab นี้ต่างจากสัปดาห์ก่อนมากขึ้น

Step 4. สร้างปุ่มรูปภาพ

เช่นปุ่มแก้ไข Profile

```
IconButton(  
    onClick = {  
        println("Edit profile")  
    }  
) {  
    Icon(  
        imageVector = Icons.Default.Edit,  
        contentDescription = "Edit"  
    )  
}
```

Flow:



5) CheckBox

Step 5. เลือก Programming Skills

สร้าง State

```
var usePython by remember {  
    mutableStateOf(false)  
}
```

แล้ว

```
Row(  
    verticalAlignment = Alignment.CenterVertically  
) {  
    Checkbox(  
        checked = usePython,  
        onCheckedChange = {  
            usePython = it  
        })  
    Text("Python")  
}
```

ทำเพิ่มอีก 2 ตัว

- ☒ Python
- ☒ Kotlin
- ☐ Java

6) ToggleButton

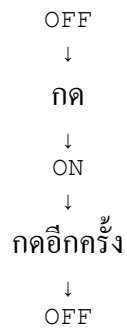
Step 6. Favorite

```
var isFavorite by remember {  
    mutableStateOf(false)  
}
```

👉

```
IconToggleButton(  
    checked = isFavorite,  
    onCheckedChange = {  
        isFavorite = it  
    }  
) {  
    Icon(  
        imageVector = Icons.Default.Favorite,  
        contentDescription = "Favorite"  
    )  
}
```

แนวคิดคือ



นี่คือ **Toggle Event + State**

7) RadioButton

Step 7. Degree

นักศึกษาต้องเลือกเพียงหนึ่งค่า

- Bachelor
- Master

สร้าง State

```
var selectedDegree by remember {  
    mutableStateOf("Bachelor")  
}
```

จากนั้น

```
RadioButton(  
    selected = selectedDegree == "Bachelor",  
    onClick = {  
        selectedDegree = "Bachelor"  
    }  
)
```

และ

```
RadioButton(  
    selected = selectedDegree == "Master",  
    onClick = {  
        selectedDegree = "Master"  
    }  
)
```

8) RadioGroup

ใน XML

```
RadioGroup  
├─ RadioButton  
└─ RadioButton
```

ส่วนใน Compose ไม่มี RadioGroup แบบเดิมที่ต้องสร้างเป็น Container เฉพาะ ดังนั้นเราจึงใช้

Column
└─ RadioButton
└─ RadioButton

แล้วใช้ State เดียวกัน

selectedDegree

เพื่อให้ได้พฤติกรรมแบบ RadioGroup

Bachelor
↓
selectedDegree

Master
↓
selectedDegree

9) Switch

Step 9. Notifications

```
var notificationEnabled by remember {  
    mutableStateOf(true)  
}
```

จากนั้น

```
Switch(  
    checked = notificationEnabled,  
    onCheckedChange = {  
        notificationEnabled = it  
    }  
)
```

แสดง

Notifications [ON]

เมื่อกด

ON → OFF
OFF → ON

ซึ่งเป็น Event Handling โดยตรง

10) SeekBar → Slider

Step 10. Interest Level

ใน View แบบเดิมเราเรียก SeekBar ส่วนใน Compose ให้ใช้ Slider() ตัวอย่าง

```
var interestLevel by remember {  
    mutableStateOf(50f)  
}
```

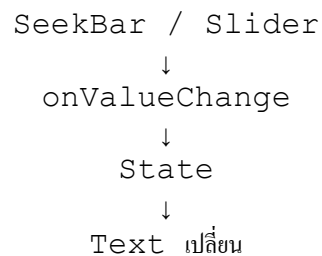
และ

```
Slider(  
    value = interestLevel,  
    onValueChange = {  
        interestLevel = it  
    },  
    valueRange = 0f..100f  
)
```

แสดง

```
Text(  
    text = "Interest Level: ${interestLevel.toInt()}%"  
)
```

ตอนลาก Slider



นี่เป็นตัวอย่างของ **Continuous Event**

11) ProgressBar → Progress Indicator

Step 11. Profile Completion

สมมติให้เริ่มที่

```
val profileProgress = 0.75f
```

สร้าง

```
LinearProgressIndicator(  
    progress = { profileProgress }  
)
```

แล้ว

```
Text("Profile Completion: 75%")
```

นักศึกษาจะเห็นความแตกต่างระหว่าง

SeekBar ที่ผู้ใช้เป็นคนลาก ส่วน **ProgressBar** App จะเป็นคนแสดงความคืบหน้า

Slider

ผู้ใช้ควบคุม



ค่าความสนใจ

Progress Indicator

App ควบคุม



ความสมบูรณ์ของ Profile

12) Button + Event

Step 12. Register

```
Button(  
    onClick = {  
        registered = true  
    }  
) {
```

```
Text("REGISTER")  
}
```

State

```
var registered by remember {  
    mutableStateOf(false)  
}
```

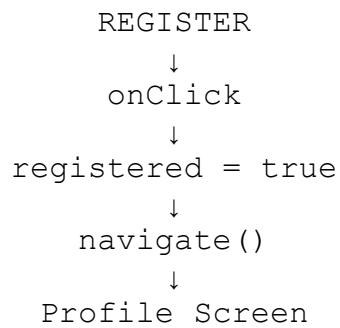
ถ้า

```
registered == true
```

ให้ Navigate ไป Profile

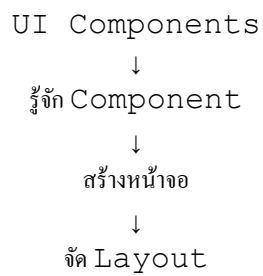
```
navController.navigate("profile")
```

ดังนั้นเกิด Event Chain



13) จุดแตกต่างจากสัปดาห์ที่ผ่านมา

Week ก่อน



Week นี้

UI Components



รับ Input



เกิด Event



เปลี่ยน State



Update UI



Navigation